

COINCRAFT RESEARCH

두 개의 장부, 왜 필요한가?

임시 운영 원장 (TypeScript) vs 영구 금융 원장 (Java)

2026.05 | 교보생명 디지털 자산 플랫폼

COINCRAFT
ACADEMY

두 장부의 역할 분리

임시 운영 원장 (TypeScript · issuer-service)

성격: 임시 · 기술적 데이터

저장 대상:

- TX in-flight 상태 추적
- 이벤트 중복 방지 키
- 발행 요청 진행 상황

수명: 30~90일 자동 만료

핵심 역할:

'지금 처리 중인 것' 추적



TX FINALIZED →
recordNftHolding()

영구 금융 원장 (Java · blockchain-gateway)

성격: 영구 · 금융 데이터

저장 대상:

- NFT 보유 현황
- 감사 로그
- 최종 정산 기록

수명: 영구 보존

핵심 역할:

'최종 확정된 것' 기록

두 장부 모두 내부망에 존재 | 구분 기준은 네트워크 위치가 아닌 역할 — TX 확정 전(임시) vs TX 확정 후(영구)

왜 분리해야 하는가?

- 1 블록체인은 확정까지 시간이 걸린다 — PENDING → MINED → FINALIZED 단계 존재
- 2 Reorg(체인 재편성) 위험 — 확정 안 된 TX가 뒤집힐 수 있다
- 3 미확정 데이터를 금융 원장에 바로 쓰면 이중 지급·잔액 오류 발생
- 4 FINALIZED 확인 후에만 Java 영구 원장에 기록 → 금융 데이터 무결성 보장

⚠ 블록체인 상태 = 확률적 확정 (Probabilistic Finality) | '블록에 포함됨' ≠ '확정됨'

각 장부의 DB 테이블

임시 운영 원장 (PostgreSQL · TypeScript)

mint_requests

TX 발행 요청 상태 추적 (PENDING→CONFIRMED) | 보존 30일

processed_events

이벤트 중복 방지 (txHash + logIndex 먹등성 키) | 보존 90일

audit_log (DMZ)

운영 레벨 감사 로그 — 시스템 접근·변경 기록

영구 금융 원장 (PostgreSQL · Java JPA)

user_nft_holdings

NFT 보유 현황 영구 기록 — 온체인 Transfer 이벤트 파생 | 보존 5년

audit_log (내부망)

금융 규제 감사 로그 — append-only · SHA-256 체크섬 | 보존 5년

audit_log는 양쪽에 존재 | DMZ = 운영 레벨 / 내부망 = 규제 대응 금융 레벨 — 목적이 다르다

T1

mint_requests — 임시 운영 원장

TX 발행 요청 상태 추적 · 보존 30일 · 임시 운영 원장 (TypeScript)

컬럼명	타입	설명
id	UUID	발행 요청 고유 식별자 (Primary Key)
user_id	VARCHAR	요청한 사용자 ID
policy_id	VARCHAR	적용된 보험 정책 ID
status	ENUM	PENDING → SUBMITTED → MINED → FINALIZED → CONFIRMED / FAILED / REJECTED
tx_hash	VARCHAR	블록체인 TX 해시 (SUBMITTED 이후 기록)
token_id	BIGINT	발행된 NFT Token ID (CONFIRMED 이후 기록)
error_msg	TEXT	실패 사유 (FAILED 시 기록)
created_at	TIMESTAMP	요청 생성 시각
updated_at	TIMESTAMP	마지막 상태 변경 시각

status 전이는 VALID_TRANSITIONS guard로만 허용 — 역방향·임의 전이 불가



processed_events — 이벤트 역등성

온체인 이벤트 중복 처리 방지 · 보존 90일 · 임시 운영 원장 (TypeScript)

컬럼명	타입	설명
id	SERIAL	자동 증가 Primary Key
tx_hash	VARCHAR	블록체인 TX 해시
log_index	INTEGER	TX 내 이벤트 순서 인덱스
event_name	VARCHAR	이벤트 이름 (예: NFTIssued)
block_number	BIGINT	이벤트 발생 블록 번호
payload	JSONB	이벤트 원본 데이터
processed_at	TIMESTAMP	이벤트 처리 시각

UNIQUE(tx_hash, log_index) — 같은 이벤트 두 번 처리 시 INSERT 무시 (ON CONFLICT DO NOTHING)

audit_log — 임시 운영 원장

운영 레벨 감사 로그 · append-only · 임시 운영 원장 (TypeScript)

컬럼명	타입	설명
id	SERIAL	자동 증가 Primary Key
event_time	TIMESTAMP	이벤트 발생 시각
actor	VARCHAR	행위자 (사용자 ID 또는 'system')
action	VARCHAR	수행 액션 (예: MINT_REQUESTED, STATUS_CONFIRMED)
resource_type	VARCHAR	대상 리소스 유형 (예: MintRequest)
resource_id	VARCHAR	대상 리소스 ID
before_state	JSONB	변경 전 상태 스냅샷 (nullable)
after_state	JSONB	변경 후 상태 스냅샷
ip_address	VARCHAR	요청 IP 주소 (nullable)
session_id	VARCHAR	세션 ID (nullable)
checksum	VARCHAR	SHA-256 무결성 체크섬

checksum = SHA-256(eventTime + actor + action + resourceId + afterState) | INSERT-only 정책 적용

user_nft_holdings — 영구 금융 원장

NFT 보유 현황 영구 기록 · 보존 5년 (가상자산이용자보호법 §15) · 영구 금융 원장 (Java)

컬럼명	타입	설명
id	BIGINT	시퀀스 Primary Key
user_id	VARCHAR	NFT 보유자 사용자 ID
token_id	BIGINT	NFT Token ID
contract_addr	VARCHAR	NFT 컨트랙트 주소 (42자 0x...)
chain_id	INTEGER	체인 ID (예: 11155111 = Sepolia)
acquired_at	TIMESTAMP	NFT 취득 시각
released_at	TIMESTAMP	NFT 소각·이전 시각 (NULL = 현재 보유 중)
on_chain_tx	VARCHAR	온체인 TX 해시 (근거 트랜잭션)

UNIQUE(token_id, contract_addr, chain_id) — 온체인이 단일 진실 | 오프체인 → 온체인 역방향 동기화 절대 금지



audit_log — 영구 금융 원장

금융 규제 감사 로그 · append-only · 보존 5년 · 영구 금융 원장 (Java)

컬럼명	타입	설명
id	BIGINT	시퀀스 Primary Key
event_time	TIMESTAMP	이벤트 발생 시각
actor	VARCHAR	행위자 (사용자 ID 또는 시스템)
action	VARCHAR	수행 액션
resource_type	VARCHAR	대상 리소스 유형
resource_id	VARCHAR	대상 리소스 ID
before_state	JSONB	변경 전 상태 스냅샷 (nullable)
after_state	JSONB	변경 후 상태 스냅샷
checksum	VARCHAR	SHA-256 무결성 체크섬

전자금융감독규정 §34 · 가상자산이용자보호법 §15 · ISMS-P | DB Row Level Security — INSERT만 허용

NFT 발행 전체 시나리오 — 단계 개요

교보생명 고객이 보험 달성 조건을 충족했을 때 NFT가 발행되는 전체 흐름

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5	STEP 6
발행 요청 생성	TX 브로드캐스트	블록 포함 확인	12블록 확정 대기	이벤트 수신·중복방지	영구 원장 기록
mint_requests INSERT (status=PENDING)	mint_requests UPDATE (SUBMITTED + tx_hash)	mint_requests UPDATE (MINED)	mint_requests UPDATE (CONFIRMED)	processed_events INSERT audit_log INSERT	mint_requests UPDATE(FINALIZED) user_nft_holdings INSERT audit_log INSERT(영구)

예외 경로

FAILED

STEP 1~5 어느 단계에서든 발생 가능
mint_requests UPDATE(status=FAILED, error_msg 기록)

REORGED

MINED 이후 체인 재편성 발생 시
mint_requests UPDATE(REORGED) → MINED 재시도

정상 경로: PENDING → SUBMITTED → MINED → CONFIRMED → FINALIZED | 종단: FINALIZED / FAILED

발행 요청 생성 & TX 브로드캐스트

STEP 1 — 보험 달성 이벤트 수신 → 발행 요청 생성

```
-- issuer-service: createMintRequest(userId, policyId)
INSERT INTO mint_requests
(id, user_id, policy_id, status, created_at, updated_at)
VALUES
('uuid-001', 'USER-홍길동', 'POL-암보험-A', 'PENDING',
now(), now());

-- audit_log INSERT: action='MINT_REQUESTED'
```

mint_requests 상태

id	user_id	status	tx_hash	token_id
uuid-001	USER- 홍길동	PENDING	NULL	NULL

STEP 2 — TX 서명 & 브로드캐스트

```
-- EVMAdapter.mintNFT() 호출 후 tx_hash 수신
UPDATE mint_requests
SET
status = 'SUBMITTED',
tx_hash = '0xabc123...def',
updated_at = now()
WHERE id = 'uuid-001';

-- audit_log INSERT: action='STATUS_SUBMITTED'
```

mint_requests 상태

id	status	tx_hash	token_id
uuid-001	SUBMITTED	0xabc1...	NULL

이 시점까지 블록체인에 TX가 전송됐을 뿐 — 아직 블록에 포함되지 않음

블록 포함 확인 & FINALIZED 대기

STEP 3 — ChainEventListener: 블록 포함 감지 (MINED)

```
-- ChainEventListener.subscribeEvents() 콜백
-- NFTIssued 이벤트 수신 → MINED 전이
UPDATE mint_requests
SET
  status      = 'MINED',
  token_id    = 42,
  updated_at  = now()
WHERE id = 'uuid-001';

-- audit_log INSERT: action='STATUS_MINED'
```

mint_requests 상태

id	status	tx_hash	token_id
uuid-001	MINED	0xabc1...	42

STEP 4 — 12블록 확정 대기 → CONFIRMED

```
-- 12블록 경과 후 Finality 확인
-- pollStaleRequests() 주기적 체크
UPDATE mint_requests
SET
  status      = 'CONFIRMED',
  updated_at  = now()
WHERE id = 'uuid-001';

-- audit_log INSERT: action='STATUS_CONFIRMED'
```

mint_requests 상태

id	status	tx_hash	token_id
uuid-001	FINALIZED	0xabc1...	42

⚠ MINED ≠ 확정 | Reorg 발생 시 MINED → REORGED → MINED 재시도 | 12블록 후에야 FINALIZED

이벤트 수신 & 중복 방지

NFTIssued 이벤트가 네트워크 지연·재시작 등으로 중복 수신될 수 있다 — processed_events가 막는다

```
-- 이벤트 수신 시 항상 먼저 중복 체크
INSERT INTO processed_events
  (tx_hash, log_index, event_name, block_number, payload)
VALUES
  ('0xabc123...', 0, 'NFTIssued', 21500000, '{"tokenId":42,...}')
ON CONFLICT (tx_hash, log_index) DO NOTHING
RETURNING id;

-- id 반환됨   → 최초 수신: 정상 처리 진행
-- id 미반환   → 중복 수신: 즉시 스킵 (이미 처리됨)
```

중복 수신 시나리오

- 1회차: id 반환 → 처리 진행
- 2회차: DO NOTHING → 스킵
- 3회차: DO NOTHING → 스킵
- 재시작 후: DO NOTHING → 스킵

processed_events 상태

tx_hash	log_index	event_name	block_number
0xabc1...	0	NFTIssued	21500000

UNIQUE(tx_hash, log_index) 제약 — 같은 이벤트는 DB 레벨에서 원천 차단 | 애플리케이션 코드 의존 없음

FINALIZED & 영구 원장 기록

CONFIRMED 확인 후 — 영구 금융 원장(Java)에 최종 기록하고 임시 원장을 FINALIZED로 닫는다

```
-- [임시 원장] mint_requests FINALIZED 전이
UPDATE mint_requests
SET status='FINALIZED', updated_at=now()
WHERE id='uuid-001';

-- [영구 원장 Java] recordNftHolding() 호출
INSERT INTO user_nft_holdings
(user_id, token_id, contract_addr, chain_id,
acquired_at, on_chain_tx)
VALUES
('USER-홍길동', 42, '0xContract...', 11155111,
now(), '0xabc123...');
```

```
-- [영구 원장] audit_log INSERT
INSERT INTO audit_log
(event_time, actor, action,
resource_type, resource_id,
before_state, after_state, checksum)
VALUES
(now(), 'system', 'NFT_HOLDING_RECORDED',
'NftHolding', '42',
NULL,
'{"userId":"USER-홍길동","tokenId":42,...}',
SHA256(...));
```

user_nft_holdings 상태 (영구 원장)

user_id	token_id	contract_addr	acquired_at	released_at
USER-홍길동	42	0xCon...	2026-05-11	NULL

FINALIZED 이후 mint_requests는 30일 후 자동 만료 | user_nft_holdings는 5년 보존

예외 시나리오 — FAILED & REORGED

FAILED — TX 실패 또는 타임아웃

```
-- 가스 부족 / RPC 오류 / 타임아웃 등
UPDATE mint_requests
SET
  status      = 'FAILED',
  error_msg   = 'insufficient gas',
  updated_at  = now()
WHERE id = 'uuid-001';

-- audit_log INSERT: action='STATUS_FAILED'
-- user_nft_holdings: INSERT 없음
-- processed_events: INSERT 없음
```

결과

- 영구 원장 기록 없음 — NFT 미발행
- 운영팀 알람 → 원인 분석 후 재발행 요청
- mint_requests 30일 후 자동 만료

REORGED — 체인 재편성 발생

```
-- MINED 상태에서 블록 재편성 감지
UPDATE mint_requests
SET
  status      = 'REORGED',
  updated_at  = now()
WHERE id = 'uuid-001';

-- TX 재브로드캐스트 → SUBMITTED로 재진입
UPDATE mint_requests
SET status='SUBMITTED', tx_hash='0xnew...'
WHERE id='uuid-001';
```

결과

- REORGED → SUBMITTED → MINED → FINALIZED 재시도
- user_nft_holdings UNIQUE(userId, tokenId) 제약으로 중복 방지
- 사용자에게는 지연만 발생, 이중 발행 없음

⚠ REORGED는 오류가 아님 — 블록체인 특성 | 자동 복구 설계가 필수